Beyond linear classifiers: feature maps and the kernel trick

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

A linear classifier draws a hyperplane, and many problems need a curved boundary. The oldest remedy keeps the linear algorithm and changes the space: map every input through a fixed non-linear feature map ϕ and learn a hyperplane in the new space. Written out explicitly, such maps quickly become enormous. The *kernel trick* avoids writing them out: any algorithm that touches the data only through dot products can be run in a very high or even infinite-dimensional feature space at the cost of evaluating a *kernel* $k(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^{\mathsf{T}}\phi(\mathbf{z})$. This note develops explicit maps, the kernel trick, the kernel perceptron, kernel ridge regression and kernel support vector machines, explains which functions are valid kernels, and states what all this costs as the number of examples grows.

1 Why linear is not enough

Note 9 studied several ways of choosing a hyperplane $\mathbf{w}^{\mathsf{T}}\mathbf{x} + b = 0$ and ended (Section 6) with the limitation they all share: the decision regions are half-spaces, so a class that lies on both sides of the other, or surrounds it, cannot be separated. The simplest example is XOR.

Worked example — 1. XOR needs one extra feature

Put the points $(0, 0)$ and $(1, 1)$ in class $t = +1$ and $(0, 1)$, $(1, 0)$ in class $t = -1$. (These are the XOR labels with the sign flipped, which changes nothing about separability.) Note 9, Section 6 showed that no line separates them.

Centre the coordinates. Put $s_j = 2x_j - 1$, so that each $s_j \in \{-1, +1\}$. The four points become $(-1, -1)$ and $(1, 1)$ in class $+1$, and $(-1, 1)$, $(1, -1)$ in class -1. The class is simply the sign of the product: $t = s_1 s_2$ on all four points.

One product feature is enough. Add the third coordinate $s_3 = s_1 s_2$. In the coordinates (s_1, s_2, s_3) the positive points become $(-1, -1, 1)$ and $(1, 1, 1)$, the negative ones $(-1, 1, -1)$ and $(1, -1, -1)$: the positives sit at height $+1$, the negatives at height -1, and the plane $s_3 = 0$ separates them. The linear function $g = s_3 = s_1 s_2$ classifies all four points correctly, with $t\, g = 1$ on each. Seen in the original plane, the boundary $s_1 s_2 = 0$ is not a single line but the pair of lines $x_1 = \frac{1}{2}$ and $x_2 = \frac{1}{2}$ (a degenerate hyperbola), and the sign of g alternates between the four quadrants they form. We shall meet this function again in Section 3, found automatically by a kernel method.

The general recipe is to choose a *feature map* $\phi : \mathbb{R}^d \to \mathbb{R}^D$ and learn a linear classifier on the mapped inputs,

$$f(\mathbf{x}) = \operatorname{sign}\left(\mathbf{w}^{\mathsf{T}}\phi(\mathbf{x}) + b\right), \qquad \mathbf{w} \in \mathbb{R}^D. \tag{1}$$

The space $\mathbb{R}^d$ of the original (raw) measurements is the *input space*; the space $\mathbb{R}^D$ of the mapped vectors, which may even be infinite-dimensional, is the *feature space*. Every algorithm of Note 9 can be used unchanged: it simply receives $\phi(\mathbf{x}_i)$ instead of $\mathbf{x}_i$. The boundary is a hyperplane in the

feature space and a curved surface in the input space. Figure 1 shows a second example: one class surrounds the other, logistic regression on $\mathbf{x}$ is at chance level (49% training error), and the same algorithm on five quadratic features separates the classes perfectly with a closed curve.

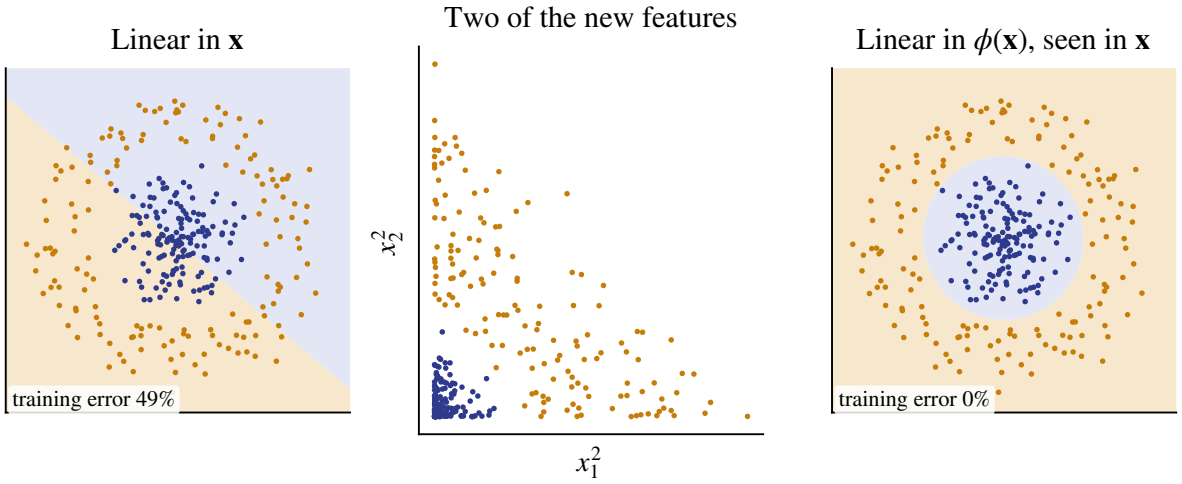


Figure 1. Left: logistic regression on the raw inputs, 300 points. Centre: two of the five quadratic features, x_1^2 and x_2^2, in which the classes are separated by a straight line. Right: logistic regression on the five features $(x_1, x_2, x_1^2, x_1x_2, x_2^2)$, with the boundary drawn in the original coordinates.

2 Explicit feature maps

Polynomial maps. The boundary in Figure 1 is close to a circle, $x_1^2 + x_2^2 = r^2$. That equation is not linear in (x_1, x_2), but it is linear in the squares: with $u_1 = x_1^2$ and $u_2 = x_2^2$ it is the straight line $u_1 + u_2 = r^2$ of the centre panel. The same holds for every conic. Write a general second-degree polynomial with coefficients c_{jk} indexed by the powers of x_1 and x_2:

$$c_{10}x_1 + c_{01}x_2 + c_{20}x_1^2 + c_{11}x_1x_2 + c_{02}x_2^2 + c_{00} = \mathbf{w}^{\mathsf{T}}\phi(\mathbf{x}) + b, \qquad \phi(\mathbf{x}) = (x_1,\ x_2,\ x_1^2,\ x_1x_2,\ x_2^2),$$

with $\mathbf{w} = (c_{10}, c_{01}, c_{20}, c_{11}, c_{02})$ and $b = c_{00}$. A polynomial in $\mathbf{x}$ is a *linear* function of its monomials, so fitting it is linear regression or linear classification on $\phi(\mathbf{x})$, and the boundary $\mathbf{w}^{\mathsf{T}}\phi(\mathbf{x}) + b = 0$ can be any conic: ellipse, parabola, hyperbola or a pair of lines. Degree-p maps give boundaries that are algebraic curves of degree at most p.

Counting the features. How large is D? A monomial $x_1^{k_1} \cdots x_d^{k_d}$ has degree $k_1 + \cdots + k_d$. Add a dummy variable $x_0 = 1$: monomials of degree *at most* p in $x_1, \ldots, x_d$ correspond one to one to monomials of degree *exactly* p in $x_0, x_1, \ldots, x_d$ (the power of x_0 makes up the difference). A monomial of degree exactly p in $d + 1$ variables is a way of placing p identical balls into $d + 1$ boxes, and there are $\binom{d+p}{p}$ of those ("stars and bars": arrange p stars and d separating bars in a row of $d + p$ symbols; the stars before the first bar are the power of x_0, those between the first and second bar the power of x_1, and so on, and a row is fixed by choosing which p of the $d + p$ positions hold stars). Hence

$$D = \binom{d+p}{p} \quad \text{monomials of degree at most } p, \text{ including the constant.} \tag{2}$$

In this note D counts the constant monomial; when a separate bias b is kept, as in Figure 1, the map itself has the $D - 1$ non-constant components. Table 1 shows how fast D grows. For fixed p it grows

like $d^p/p!$; a 28×28 image ($d = 784$) with cubic features needs eighty million of them. One sometimes reads the cruder estimate d^p: that is the number of *ordered* products $x_{j_1} x_{j_2} \cdots x_{j_p}$, which counts $x_1 x_2$ and $x_2 x_1$ separately. For $d = 100$ and $p = 5$ it gives 10^{10}, while merging equal monomials divides it by roughly $p! = 120$ and leaves the $\approx 10^8$ of Table 1. The factors $\sqrt{2}$ in Worked example 3 are what remains of this merging.

Table 1. Number of polynomial features $D = \binom{d+p}{p}$ (constant included).

	$p = 2$	$p = 3$	$p = 5$
$d = 2$	6	10	21
$d = 10$	66	286	3 003
$d = 100$	5 151	176 851	96 560 646
$d = 784$	308 505	80 931 145	$\approx 2.5 \times 10^{12}$

What explicit maps cost. Three things grow with D. *Memory*: storing $\phi(\mathbf{x}_i)$ for n examples takes nD numbers; for $n = 10^4$, $d = 100$, $p = 3$ that is $1.8{\times}10^9$ numbers, about 7 GB in 32-bit floats. *Time*: every gradient step of logistic regression or every pass of the perceptron touches all nD numbers. *Capacity*: the model has D parameters (the constant monomial plays the role of the bias; the quadratic map in the plane has $D = 6$: five features plus the bias) and can overfit, so regularisation (Note 4, Section 7) and more data are needed — the curse of dimensionality (Note 3, Section 3) in a new form.

Polynomials are not the only explicit choice. Physically motivated variables (an invariant mass, an energy, a radius $\|\mathbf{x}\|$), Fourier features $\sin(\boldsymbol{\omega}^\mathsf{T}\mathbf{x})$ and $\cos(\boldsymbol{\omega}^\mathsf{T}\mathbf{x})$, and random non-linear features (the random ReLU features of Note 4, Section 8) are all feature maps. Algorithm 1 is the same for all of them.

Algorithm 1 Explicit feature map followed by a linear classifier

Require: training set $\{(\mathbf{x}_i, t_i)\}_{i=1}^n$, $\mathbf{x}_i \in \mathbb{R}^d$, $t_i \in \{-1, +1\}$; feature map $\phi : \mathbb{R}^d \to \mathbb{R}^D$; a linear training algorithm LinearFit (any of Note 9)

function ExplicitFit($\{(\mathbf{x}_i, t_i)\}$)
  for $i = 1, \dots, n$ **do**
    row i of $\Phi \leftarrow \phi(\mathbf{x}_i)^\mathsf{T}$ ▷ $O(D)$ for a polynomial map
  end for
  $(\mathbf{w}, b) \leftarrow$ LinearFit($\Phi, \mathbf{t}$) ▷ e.g. perceptron, logistic regression, linear SVM
  return $(\mathbf{w}, b)$
end function
function ExplicitPredict($\mathbf{x}$; $\mathbf{w}, b$)
  return $\operatorname{sign}\left(\mathbf{w}^\mathsf{T}\phi(\mathbf{x}) + b\right)$
end function

Running time — explicit feature map + linear classifier

Training: computing the features costs $O(nD)$ (each monomial is one product of a lower-degree monomial and one coordinate); T passes of gradient descent or of the perceptron cost $O(TnD)$; a closed-form least-squares classifier costs $O(nD^2 + D^3)$ (forming and solving the $D \times D$ normal equations, Note 8). *Prediction*: $O(D)$ per query, for $\phi(\mathbf{x})$ and one dot product. *Memory*: $O(nD)$ for the mapped data, or $O(D)$ if features are recomputed on the fly in each pass. With $D = \binom{d+p}{p}$ this is fine up to $D \sim 10^5$ and hopeless beyond 10^7.

Worked example — 2. A perceptron on explicit quadratic features

Three points on a line, $x = -1, 0, 2$, with labels $t = +1, -1, +1$: the negative point sits between the two positive ones, so no single threshold on x separates them. Map each point to $\phi(x) = (x, x^2)$, giving $(-1, 1)$, $(0, 0)$ and $(2, 4)$, and run the perceptron of Note 9, Section 3, with a bias: start from $\mathbf{w} = (0, 0)$, $b = 0$, and on a mistake ($t\, g \le 0$ with $g = \mathbf{w}^{\mathsf{T}}\phi(x) + b$) set $\mathbf{w} \leftarrow \mathbf{w} + t\,\phi(x)$, $b \leftarrow b + t$.

pass	x	t	$\phi(x)$	g (before update)	result	$(\mathbf{w};\ b)$ after
1	-1	$+1$	$(-1, 1)$	0	mistake	$(-1, 1;\ 1)$
1	0	-1	$(0, 0)$	1	mistake	$(-1, 1;\ 0)$
1	2	$+1$	$(2, 4)$	$-2 + 4 + 0 = 2$	correct	
2	-1	$+1$	$(-1, 1)$	$1 + 1 + 0 = 2$	correct	
2	0	-1	$(0, 0)$	0	mistake	$(-1, 1;\ -1)$
2	2	$+1$	$(2, 4)$	1	correct	
3	all			$1, -1, 1$	all correct	converged

After three mistakes the perceptron stops with $\mathbf{w} = (-1, 1)$ and $b = -1$, a straight line in the (x, x^2) plane. Read in the input space it is the quadratic

$$g(x) = x^2 - x - 1,$$

whose roots $x = (1 \pm \sqrt{5})/2 \approx -0.618$ and 1.618 are the decision boundary: points between them are classified -1, points outside $+1$. The algorithm never knew it was fitting a parabola; it saw only two-dimensional vectors. Question B3 runs the kernel version of the perceptron on a similar problem.

Three routes to non-linearity. The feature map can be fixed in advance and computed, fixed in advance but never computed, or fitted to the data. The routes differ in what the designer chooses, what training adjusts, and what the cost grows with:

route	chosen by hand	fitted to data	cost grows with
explicit map (this section)	ϕ itself	$\mathbf{w} \in \mathbb{R}^D$	D, the number of features
kernel (Sections 3–8)	$k(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^{\mathsf{T}}\phi(\mathbf{z})$	n coefficients α_i	n, through an $n \times n$ matrix
learned features (week 5)	the form of ϕ, e.g. $a(V\mathbf{x} + \mathbf{c})$	V, $\mathbf{c}$ and $\mathbf{w}$ together	the number of parameters; non-convex training

In the third row a is an element-wise non-linearity and V a matrix trained by gradient descent (Note 7) with $\mathbf{w}$: a multilayer perceptron is a linear model on features that are themselves learned.

3 The kernel trick

3.1 Algorithms that need only dot products

Note 9, Section 3 ("The dual form") showed that the perceptron's weight vector is a sum of the training inputs weighted by mistake counts. Here we drop the bias (it returns below as $k + 1$) and run the same algorithm on the mapped inputs $\phi(\mathbf{x}_i)$. It starts from $\mathbf{w} = \mathbf{0}$ and, on each mistake on

example i, adds $t_i\phi(\mathbf{x}_i)$ to $\mathbf{w}$. After training, therefore,

$$\mathbf{w} = \sum_{i=1}^{n} \alpha_i t_i\, \phi(\mathbf{x}_i), \qquad \alpha_i = \text{number of mistakes made on example } i. \tag{3}$$

The weight vector is a combination of the training inputs, and the n integers α_i describe it completely, whatever the dimension D. Its score on any input is

$$g(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}\phi(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i t_i\, \phi(\mathbf{x}_i)^{\mathsf{T}}\phi(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i t_i\, k(\mathbf{x}_i, \mathbf{x}), \tag{4}$$

where the *kernel*

$$k(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^{\mathsf{T}}\phi(\mathbf{z}) \tag{5}$$

is the dot product in feature space. Training needs the scores $g(\mathbf{x}_j)$ on training points, hence only the numbers $k(\mathbf{x}_i, \mathbf{x}_j)$, collected in the $n \times n$ *Gram matrix* $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$; prediction needs only $k(\mathbf{x}_i, \mathbf{x})$. If k can be evaluated without computing ϕ, the algorithm runs in feature space without ever visiting it. That is the kernel trick [1, 6].

Why the weight vector lies in the span of the data. Equation (3) is not a peculiarity of the perceptron. Consider any objective of the form

$$J(\mathbf{w}) = L\big(\mathbf{w}^{\mathsf{T}}\phi(\mathbf{x}_1), \dots, \mathbf{w}^{\mathsf{T}}\phi(\mathbf{x}_n)\big) + \lambda \left\|\mathbf{w}\right\|^2, \qquad \lambda > 0,$$

where L is any loss that depends on $\mathbf{w}$ only through the scores on the training points — squared error, logistic loss, hinge loss. Split $\mathbf{w} = \mathbf{w}_{\parallel} + \mathbf{w}_{\perp}$, with $\mathbf{w}_{\parallel}$ in the span of $\phi(\mathbf{x}_1), \dots, \phi(\mathbf{x}_n)$ and $\mathbf{w}_{\perp}$ orthogonal to all of them. Then $\mathbf{w}^{\mathsf{T}}\phi(\mathbf{x}_i) = \mathbf{w}_{\parallel}^{\mathsf{T}}\phi(\mathbf{x}_i)$ for every i, so L does not see $\mathbf{w}_{\perp}$, while $\left\|\mathbf{w}\right\|^2 = \left\|\mathbf{w}_{\parallel}\right\|^2 + \left\|\mathbf{w}_{\perp}\right\|^2$ is smallest for $\mathbf{w}_{\perp} = \mathbf{0}$. The minimiser therefore has the form $\mathbf{w} = \sum_i \beta_i \phi(\mathbf{x}_i)$. This is the *representer theorem* [12, 20]; it covers ridge regression, regularised logistic regression and the SVM.

Kernelising in practice. Every method in this note is obtained the same way. Examples must meet only in pairs, through $\mathbf{x}_i^{\mathsf{T}}\mathbf{x}_j$ during training and $\mathbf{x}_i^{\mathsf{T}}\mathbf{x}$ at prediction; the algorithm keeps track of the n coefficients of $\mathbf{w}$ in the span of the data instead of $\mathbf{w}$ itself; and each pairwise dot product is evaluated as $k(\mathbf{x}_i, \mathbf{x}_j)$. Nothing is approximated: the kernelised algorithm produces exactly the iterates that the original one would produce on the points $\phi(\mathbf{x}_i)$. The mapping must happen *before* the algorithm runs, because ϕ is non-linear and does not commute with taking combinations: $\sum_i \beta_i \phi(\mathbf{x}_i)$ is in general not the image $\phi(\mathbf{v})$ of any input point $\mathbf{v}$, let alone of $\sum_i \beta_i \mathbf{x}_i$ (Worked example 4 shows a case).

3.2 Computing a dot product without the features

Worked example — 3. Two polynomial kernels, checked by hand

The homogeneous quadratic kernel. For $\mathbf{x}, \mathbf{z} \in \mathbb{R}^2$ expand

$$(\mathbf{x}^{\mathsf{T}}\mathbf{z})^2 = (x_1 z_1 + x_2 z_2)^2 = x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2 = \phi(\mathbf{x})^{\mathsf{T}}\phi(\mathbf{z}), \qquad \phi(\mathbf{x}) = \big(x_1^2,\ \sqrt{2}\, x_1 x_2,\ x_2^2\big).$$

The factor $\sqrt{2}$ appears because the cross term occurs twice. Take $\mathbf{x} = (1, 2)$ and $\mathbf{z} = (3, 1)$. With the kernel: $\mathbf{x}^{\mathsf{T}}\mathbf{z} = 3 + 2 = 5$, so $k(\mathbf{x}, \mathbf{z}) = 25$. With the features,

$$\phi(\mathbf{x}) = (1,\ 2\sqrt{2},\ 4), \quad \phi(\mathbf{z}) = (9,\ 3\sqrt{2},\ 1), \quad \phi(\mathbf{x})^{\mathsf{T}}\phi(\mathbf{z}) = 9 + 2\sqrt{2}\cdot 3\sqrt{2} + 4 = 9 + 12 + 4 = 25.$$

The inhomogeneous quadratic kernel. Expanding $(1 + \mathbf{x}^\mathsf{T}\mathbf{z})^2 = 1 + 2\mathbf{x}^\mathsf{T}\mathbf{z} + (\mathbf{x}^\mathsf{T}\mathbf{z})^2$ gives a six-dimensional map,

$$\phi(\mathbf{x}) = \big(1,\ \sqrt{2}\,x_1,\ \sqrt{2}\,x_2,\ x_1^2,\ \sqrt{2}\,x_1x_2,\ x_2^2\big),$$

which contains all $\binom{2+2}{2} = 6$ monomials of degree at most two, with weights. For the same points the kernel gives $k(\mathbf{x}, \mathbf{z}) = (1 + 5)^2 = 36$, while

$$\phi(\mathbf{x}) = (1, \sqrt{2}, 2\sqrt{2}, 1, 2\sqrt{2}, 4), \qquad \phi(\mathbf{z}) = (1, 3\sqrt{2}, \sqrt{2}, 9, 3\sqrt{2}, 1),$$

and the six products sum to $\phi(\mathbf{x})^\mathsf{T}\phi(\mathbf{z}) = 1 + 6 + 4 + 9 + 12 + 4 = 36$.
The costs. The kernel needs one d-dimensional dot product, one addition and one squaring: about $d + 2$ operations. The features need D operations for each ϕ and D more for the dot product. For $(1+\mathbf{x}^\mathsf{T}\mathbf{z})^p$ with $d = 100$ and $p = 3$, that is about 10^2 operations against $3{\times}176\,851 \approx 5 \times 10^5$, a factor of several thousand, and the gap widens with p.

The same expansion works in general: by the multinomial theorem, $(c + \mathbf{x}^\mathsf{T}\mathbf{z})^p$ with $c > 0$ is the dot product of two vectors containing all $\binom{d+p}{p}$ monomials of degree at most p, each multiplied by the square root of a positive coefficient; $(\mathbf{x}^\mathsf{T}\mathbf{z})^p$ contains only the monomials of degree exactly p. A degree-p polynomial classifier can thus be trained at cost $O(d)$ per kernel evaluation instead of $O(d^p)$.

3.3 The kernel perceptron

Combining (3) and (4) turns the perceptron into an algorithm on the mistake counts α_i [1, 9]. The mistake test for example j, $t_j\,\mathbf{w}^\mathsf{T}\phi(\mathbf{x}_j) \le 0$, becomes $t_j \sum_i \alpha_i t_i K_{ij} \le 0$, and the update $\mathbf{w} \leftarrow \mathbf{w} + t_j\phi(\mathbf{x}_j)$ becomes $\alpha_j \leftarrow \alpha_j + 1$.

A bias for free. The linear perceptron also updates a bias, $b \leftarrow b + t_j$. The bias starts at 0 and gains t_i on every mistake on example i, so in the kernel version it equals $b = \sum_i \alpha_i t_i$, and $g(\mathbf{x}) + b = \sum_i \alpha_i t_i\,[k(\mathbf{x}_i, \mathbf{x}) + 1]$: keeping a bias is the same as replacing k by $k + 1$, which appends a constant feature to ϕ. The kernel $(1 + \mathbf{x}^\mathsf{T}\mathbf{z})^p$ already contains one.

Running time — kernel perceptron

Training: the Gram matrix costs $O(n^2 d)$ once (n^2 kernel values at $O(d)$ each); each score g_j is a sum over at most n terms, so one pass costs $O(n^2)$ and T passes $O(n^2(d+T))$. *Memory*: $O(n^2)$ for K; without storing K, recompute kernel values on demand, $O(nd)$ memory and $O(Tn\,Md)$ time, where M is the number of examples with $\alpha_i > 0$. *Prediction*: $O(Md)$ per query, one kernel evaluation per stored example. If the mapped data are separable with margin γ by a hyperplane through the origin of feature space (or, when a bias is kept, with k replaced by $k+1$), Novikoff's bound (Note 9, Section 3) applies with $R^2 = \max_i k(\mathbf{x}_i, \mathbf{x}_i)$: the total number of mistakes — and hence M — is at most $(R/\gamma)^2$, independent of D.

Worked example — 4. The kernel perceptron solves XOR

Use the centred XOR points of Section 1 and the kernel $k(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}^\mathsf{T}\mathbf{z})^2$. From now on we write $\mathbf{x} = (x_1, x_2)$ for the centred coordinates (s_1, s_2) of Worked example 1; bold $\mathbf{x}_1, \dots, \mathbf{x}_4$ label the four points.

	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$
$\mathbf{x}$	$(1, 1)$	$(-1, -1)$	$(1, -1)$	$(-1, 1)$
t	$+1$	$+1$	-1	-1

Algorithm 2 Kernel perceptron

Require: training set $\{(\mathbf{x}_i, t_i)\}_{i=1}^{n}$, $t_i \in \{-1, +1\}$; kernel k; maximum number of passes T
Ensure: mistake counts $\alpha_1, \dots, \alpha_n$

function KernelPerceptronFit($\{(\mathbf{x}_i, t_i)\}$, k, T)
  $K \leftarrow \textsc{Gram}(\mathbf{x}_1, \dots, \mathbf{x}_n; k)$ ▷ Algorithm 3, $O(n^2 d)$
  $\alpha_i \leftarrow 0$ for all i
  for pass $= 1, \dots, T$ **do**
    mistakes $\leftarrow 0$
    for $j = 1, \dots, n$ **do** ▷ or in random order
      $g_j \leftarrow \sum_{i:\alpha_i>0} \alpha_i t_i K_{ij}$ ▷ score of $\mathbf{x}_j$, $O(n)$
      if $t_j g_j \le 0$ **then**
        $\alpha_j \leftarrow \alpha_j + 1$; mistakes $\leftarrow$ mistakes $+1$
      end if
    end for
    if mistakes $= 0$ **then break**
    end if
  end for
  return $\boldsymbol{\alpha}$
end function
function KernelPerceptronPredict($\mathbf{x}$; $\{(\mathbf{x}_i, t_i, \alpha_i)\}$, k)
  return $\operatorname{sign}\big(\sum_{i:\alpha_i>0} \alpha_i t_i\, k(\mathbf{x}_i, \mathbf{x})\big)$
end function

Gram matrix. Each point has $\mathbf{x}_i^{\mathsf{T}}\mathbf{x}_i = 2$, so $K_{ii} = (1+2)^2 = 9$. Neighbouring points have $\mathbf{x}_i^{\mathsf{T}}\mathbf{x}_j = 0$ and opposite points $\mathbf{x}_i^{\mathsf{T}}\mathbf{x}_j = -2$; both give $K_{ij} = 1$. Hence $K = 8I + \mathbf{1}\mathbf{1}^{\mathsf{T}}$: nines on the diagonal, ones elsewhere, and the score of $\mathbf{x}_j$ is

$$g_j = \sum_i \alpha_i t_i K_{ij} = 8\,\alpha_j t_j + S, \qquad S = \sum_i \alpha_i t_i.$$

Trace. Visit the points in the order $1, 2, 3, 4$ and update when $t_j g_j \le 0$. The column g_j shows the two terms $8\alpha_j t_j$ and S, evaluated before the update:

pass	j	t_j	$g_j = 8\alpha_j t_j + S$	$t_j g_j \le 0$?	$\boldsymbol{\alpha}$ after	
1	1	$+1$	0	mistake	$(1,0,0,0)$	start: all scores zero
1	2	$+1$	$0 + 1 = 1$	no	$(1,0,0,0)$	
1	3	-1	$0 + 1 = 1$	mistake	$(1,0,1,0)$	S becomes 0
1	4	-1	$0 + 0 = 0$	mistake	$(1,0,1,1)$	S becomes -1
2	1	$+1$	$8 - 1 = 7$	no		
2	2	$+1$	$0 - 1 = -1$	mistake	$(1,1,1,1)$	S becomes 0
2	3	-1	$-8 + 0 = -8$	no		
2	4	-1	$-8 + 0 = -8$	no		
3	1–4		$8, 8, -8, -8$	no	$(1,1,1,1)$	converged

After four mistakes every point is classified correctly, with $\boldsymbol{\alpha} = (1,1,1,1)$.
The decision function. Put $u = x_1 + x_2$ and $v = x_1 - x_2$. Then $\mathbf{x}_1^{\mathsf{T}}\mathbf{x} = u$, $\mathbf{x}_2^{\mathsf{T}}\mathbf{x} = -u$, $\mathbf{x}_3^{\mathsf{T}}\mathbf{x} = v$ and $\mathbf{x}_4^{\mathsf{T}}\mathbf{x} = -v$, so with all $\alpha_i = 1$ the score $g(\mathbf{x}) = \sum_i \alpha_i t_i (1 + \mathbf{x}_i^{\mathsf{T}}\mathbf{x})^2$ is

$$g(\mathbf{x}) = (1+u)^2 + (1-u)^2 - (1+v)^2 - (1-v)^2 = 2u^2 - 2v^2 = 8\,x_1 x_2.$$

The kernel perceptron has found the sign of the product, the function of Section 1, without ever forming a product feature. At a new point, $g(0.5, 0.5) = 2$ and $g(2, -1) = -16$.
What happened in feature space. With the six features of Worked example 3, $\mathbf{w} = \sum_i \alpha_i t_i \phi(\mathbf{x}_i)$: the constant, linear and squared components cancel in pairs and only the cross feature survives, $\mathbf{w} = (0, 0, 0, 0, 4\sqrt{2}, 0)$, so $\mathbf{w}^\mathsf{T}\phi(\mathbf{x}) = 4\sqrt{2} \cdot \sqrt{2}\, x_1 x_2 = 8 x_1 x_2$. Combining the inputs first and mapping afterwards gives something else entirely: $\sum_i \alpha_i t_i \mathbf{x}_i = (0, 0)$ and $\phi(0, 0) = (1, 0, 0, 0, 0, 0)$, a constant classifier. In fact no input $\mathbf{v}$ has $\phi(\mathbf{v}) = \mathbf{w}$, since the first component of $\phi(\mathbf{v})$ is always 1.
The mistake bound. Every point has $\|\phi(\mathbf{x}_i)\|^2 = K_{ii} = 9$, so $R = 3$. The unit vector along the cross feature, a hyperplane through the origin of feature space, separates the data with margin $t_i\sqrt{2}\, x_{i1} x_{i2} = \sqrt{2}$. Novikoff's bound allows at most $(R/\gamma)^2 = 9/2$ mistakes; we made four.

Computing the Gram matrix. Every kernel method starts by filling K. For the Gaussian kernel of the next section, the identity $\|\mathbf{x}_i - \mathbf{x}_j\|^2 = \|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2 - 2\mathbf{x}_i^\mathsf{T}\mathbf{x}_j$ (Note 1, Section 6) turns all squared distances into one matrix product XX^T.

Algorithm 3 Gram matrix (vectorised)

Require: data matrix $X \in \mathbb{R}^{n \times d}$ (rows $\mathbf{x}_i^\mathsf{T}$), kernel k

function GRAM(X; k)
  $P \leftarrow XX^\mathsf{T}$ ▷ all dot products, $O(n^2 d)$
  if k is polynomial, $(c + \mathbf{x}^\mathsf{T}\mathbf{z})^p$ **then**
    $K \leftarrow (c + P)^{\circ p}$ ▷ element-wise power
  else if k is Gaussian with width σ **then**
    $\mathbf{q} \leftarrow \mathrm{diag}(P)$ ▷ squared norms $\|\mathbf{x}_i\|^2$
    $K_{ij} \leftarrow \exp\big(-(q_i + q_j - 2P_{ij})/(2\sigma^2)\big)$ for all i, j
  else
    $K_{ij} \leftarrow k(\mathbf{x}_i, \mathbf{x}_j)$ for all $i \le j$; $K_{ji} \leftarrow K_{ij}$ ▷ any other kernel
  end if
  return K ▷ symmetric: only $n(n+1)/2$ distinct entries
end function

Running time — Gram matrix

Time: $O(n^2 d)$, dominated by XX^T, a single matrix product that BLAS libraries and GPUs execute at close to peak speed; the element-wise step adds $O(n^2)$. *Memory*: $O(n^2)$: 10^8 numbers or 0.8 GB in 64-bit floats for $n = 10^4$; 80 GB for $n = 10^5$; 8 TB for $n = 10^6$. Memory, not time, usually limits exact kernel methods. A test block $K(X_{\text{test}}, X)$ costs $O(n_{\text{test}}\, n\, d)$.

4 Common kernels

Linear. $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\mathsf{T}\mathbf{z}$, with $\phi(\mathbf{x}) = \mathbf{x}$. It reproduces the original linear algorithm; kernelising with it is useful only when $d > n$, where the $n \times n$ Gram matrix is smaller than the $d \times d$ matrices of the primal form.

Polynomial. $k(\mathbf{x}, \mathbf{z}) = (c + \mathbf{x}^\mathsf{T}\mathbf{z})^p$ with $c \ge 0$ and integer $p \ge 1$: all monomials of degree at most p ($c > 0$) or exactly p ($c = 0$). The constant c sets the weight of low-degree terms relative to high-degree ones. For large inputs k grows like $(\|\mathbf{x}\|\,\|\mathbf{z}\|)^p$ (so $k(\mathbf{x}, \mathbf{x}) \sim \|\mathbf{x}\|^{2p}$), and a large p quickly produces huge

or tiny numbers, so inputs must be scaled to modest norms. The degree p and the constant c are hyperparameters, chosen together with the regularisation constant by validation (Note 4, Section 7).

Gaussian or radial basis function (RBF).

$$k_\sigma(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2}\right). \tag{6}$$

It equals 1 when $\mathbf{x} = \mathbf{z}$ and falls to 0 as the points move apart, on the length scale σ: a similarity measure. It depends only on $\mathbf{x} - \mathbf{z}$ (it is *stationary*, or translation invariant), and is by far the most common default. Its feature space is infinite-dimensional.

Worked example — 5. The Gaussian kernel has infinitely many features

In one dimension, factor the exponent: $-(x-z)^2/(2\sigma^2) = -x^2/(2\sigma^2) - z^2/(2\sigma^2) + xz/\sigma^2$, so

$$k_\sigma(x, z) = e^{-x^2/2\sigma^2}\, e^{-z^2/2\sigma^2}\, e^{xz/\sigma^2} = e^{-x^2/2\sigma^2}\, e^{-z^2/2\sigma^2} \sum_{m=0}^{\infty} \frac{1}{m!} \left(\frac{xz}{\sigma^2}\right)^m,$$

using the Taylor series $e^u = \sum_m u^m/m!$. Each term splits into a function of x times the same function of z:

$$k_\sigma(x, z) = \sum_{m=0}^{\infty} \phi_m(x)\, \phi_m(z), \qquad \phi_m(x) = e^{-x^2/2\sigma^2}\, \frac{x^m}{\sigma^m \sqrt{m!}}.$$

The feature map $\phi(x) = (\phi_0(x), \phi_1(x), \phi_2(x), \dots)$ has infinitely many components: every power of x, damped by a Gaussian envelope and by $1/\sqrt{m!}$.
Numerically. Take $x = 1$, $z = 0.5$, $\sigma = 1$. Exactly, $k = e^{-0.125} = 0.8825$. The prefactor is $e^{-0.5}e^{-0.125} = 0.5353$, and keeping the terms $m = 0, 1, \dots, m_{\max}$ of the series gives

$m_{\max}$	0	1	2	3	4	5
partial sum	0.5353	0.8029	0.8698	0.8810	0.8823	0.8825

The series converges quickly here, but no finite truncation is exact for all x, z, and the kernel evaluates the infinite sum in one exponential. In d dimensions k_σ is the product of d such one-dimensional kernels, because $\|\mathbf{x} - \mathbf{z}\|^2 = \sum_j (x_j - z_j)^2$ and the exponential of a sum is the product of the exponentials. A product of kernels has as features all pairwise products of their features (Section 5, rule 3), so the features of k_σ are all monomials of all degrees, each with its Gaussian envelope.

The width σ. The width is the most important hyperparameter of the Gaussian kernel. As $\sigma \to 0$, $K \to I$: each training point is similar only to itself, the model can memorise any labelling, and the boundary breaks into small islands around individual points. As $\sigma \to \infty$, all kernel values approach 1 and differ only through the small term $-\|\mathbf{x} - \mathbf{z}\|^2/(2\sigma^2)$. In the Taylor series of Worked example 5 the terms of degree m carry a factor σ^{-2m}, so only the first few powers matter: the model behaves like a low-degree polynomial classifier and cannot follow a strongly curved boundary. Figure 2 shows both extremes and a good compromise on the two-moons data (the classifier used there, the kernel SVM, comes in Section 7). Choose σ, together with the regularisation constant, by cross-validation (Note 4, Sections 4 and 7); a common starting point is the median distance between training points. Because the kernel depends on Euclidean distances, standardise the features first (Note 5, Section 5).

Many other kernels exist: the Laplacian kernel $\exp(-\|\mathbf{x} - \mathbf{z}\|/\sigma)$, kernels that compare strings by their common substrings (used for DNA and protein sequences), kernels on trees (for instance the

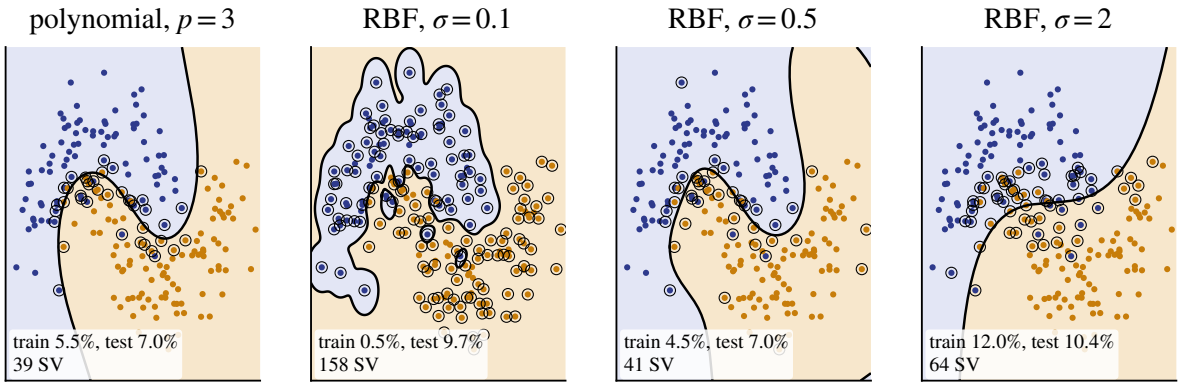


Figure 2. Kernel SVMs ($C = 10$) on 200 points of the two-moons data, trained by Algorithm 5 (bias absorbed by adding 1 to the kernel, as in "A bias for free"); test errors on 2000 new points. The kernel SVM, its constant C and its support vectors (circled) are explained in Section 7; here only the shape of the boundary matters. A cubic polynomial kernel, $(1 + \mathbf{x}^\mathsf{T}\mathbf{z})^3$, and a Gaussian kernel of width $\sigma = 0.5$ both reach 7.0% test error. The narrow Gaussian ($\sigma = 0.1$) fits the training set almost perfectly (0.5%) but draws islands, uses 158 of the 200 points as support vectors and reaches 9.7% test error; the wide one ($\sigma = 2$) is too smooth (12.0% training, 10.4% test error).

parse trees of sentences in language processing), on graphs and on sets. What they share is the property of the next section.

5 What makes a valid kernel

A function $k(\mathbf{x}, \mathbf{z})$ is a *kernel* if there exists a feature map ϕ, into $\mathbb{R}^D$ or into an infinite-dimensional space with a dot product, such that $k(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\mathsf{T}\phi(\mathbf{z})$ for all $\mathbf{x}, \mathbf{z}$. Not every similarity function qualifies.

A necessary condition. Take any points $\mathbf{x}_1, \dots, \mathbf{x}_n$ and any real coefficients $c_1, \dots, c_n$. If k is a kernel,

$$\mathbf{c}^\mathsf{T} K \mathbf{c} = \sum_{i,j} c_i c_j \, \phi(\mathbf{x}_i)^\mathsf{T}\phi(\mathbf{x}_j) = \Big(\sum_i c_i \phi(\mathbf{x}_i) \Big)^\mathsf{T} \Big(\sum_j c_j \phi(\mathbf{x}_j) \Big) = \Big\| \sum_i c_i \phi(\mathbf{x}_i) \Big\|^2 \geq 0.$$

So every Gram matrix of a kernel is symmetric and *positive semi-definite* (PSD): $\mathbf{c}^\mathsf{T} K \mathbf{c} \geq 0$ for all $\mathbf{c}$, equivalently all its eigenvalues are non-negative. A squared length cannot be negative.

The converse (Moore–Aronszajn, Mercer). The condition is also sufficient: a symmetric function k for which every Gram matrix, on any finite set of points, is PSD is a kernel for some feature map ϕ. In this form the result is the reproducing-kernel Hilbert-space construction usually credited to Moore and Aronszajn; Mercer's 1909 theorem is the continuous version, for continuous k on a compact domain, with ϕ built from the eigenfunctions of k [13, 21]. (These constructions are beyond this course; all we use is the statement: a symmetric k is a valid kernel exactly when all its Gram matrices are PSD.) Kernels with this property are therefore often called Mercer kernels. This is why one can use the Gaussian kernel without ever writing its features: it is enough to know that its Gram matrices are PSD.

Worked example — 6. Checking a Gram matrix

A valid kernel. Take the one-dimensional points $x = 0, 1, 2$ and $k(x, z) = (1 + xz)^2$:

$$K = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 4 & 9 \\ 1 & 9 & 25 \end{pmatrix}.$$

It is symmetric. Its *leading principal minors* are the determinants of its top-left 1×1, 2×2 and 3×3 sub-matrices; for a symmetric matrix, if all of them are positive the matrix is positive definite (Sylvester's criterion), and hence PSD. Here $1 > 0$, $1 \cdot 4 - 1 \cdot 1 = 3 > 0$ and $\det K = 1(100 - 81) - 1(25 - 9) + 1(9 - 4) = 19 - 16 + 5 = 8 > 0$. Numerically the eigenvalues are 0.200, 1.411 and 28.390, all positive, as they must be. (The test only works in this direction: a minor equal to zero does not settle the question, and then one checks the eigenvalues instead.)
Not a kernel. The squared distance $k(\mathbf{x}, \mathbf{z}) = \|\mathbf{x} - \mathbf{z}\|^2$ is symmetric and non-negative, a reasonable "dissimilarity". On the two points 0 and 1 its Gram matrix is $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, and with $\mathbf{c} = (1, -1)$, $\mathbf{c}^\mathsf{T} K \mathbf{c} = 0 - 1 - 1 + 0 = -2 < 0$: it would be a vector of negative squared length. Its eigenvalues are ± 1. (Its negative, $-\|\mathbf{x} - \mathbf{z}\|^2$, fails too, with $\mathbf{c} = (1, 1)$.)

A non-PSD "kernel" is not only a theoretical flaw: the dual SVM problem then stops being concave, so solvers can get stuck in local maxima (and for the hard margin the objective can be unbounded), and $K + \lambda I$ in kernel ridge regression can become singular. The sigmoid function $\tanh(a\, \mathbf{x}^\mathsf{T}\mathbf{z} + c)$, offered as a kernel in early SVM software, is a well-known example: it is not positive semi-definite in general, although its Gram matrices happen to be PSD for some parameter values on some data sets [21].

Building kernels from kernels. Rather than check PSD-ness each time, build new kernels from known ones. If k_1 and k_2 are kernels with feature maps ϕ_1 and ϕ_2, so are

1. $k_1 + k_2$: concatenate the features, $\phi = (\phi_1, \phi_2)$;
2. $c\, k_1$ for $c > 0$: scale the features, $\phi = \sqrt{c}\, \phi_1$;
3. $k_1 k_2$: take all pairwise products of features, $\phi_{ab}(\mathbf{x}) = \phi_{1,a}(\mathbf{x})\, \phi_{2,b}(\mathbf{x})$; then $\sum_{a,b} \phi_{ab}(\mathbf{x})\phi_{ab}(\mathbf{z}) = \big(\sum_a \phi_{1,a}(\mathbf{x})\phi_{1,a}(\mathbf{z})\big)\big(\sum_b \phi_{2,b}(\mathbf{x})\phi_{2,b}(\mathbf{z})\big) = k_1 k_2$;
4. $f(\mathbf{x})\, k_1(\mathbf{x}, \mathbf{z})\, f(\mathbf{z})$ for any real function f: $\phi = f\phi_1$;
5. any polynomial in k_1 with non-negative coefficients (a constant $c \geq 0$ is itself a kernel, with the single feature $\sqrt{c}$), and $\exp(k_1)$, the limit of its Taylor polynomials: by rules 1–3, and because a pointwise limit of kernels is a kernel (a limit of PSD matrices is PSD);
6. $k_1(\psi(\mathbf{x}), \psi(\mathbf{z}))$ for any map ψ, for instance a preprocessing step.

For example, $(1 + \mathbf{x}^\mathsf{T}\mathbf{z})^2 = 1 + 2\, \mathbf{x}^\mathsf{T}\mathbf{z} + (\mathbf{x}^\mathsf{T}\mathbf{z})^2$ is valid by rules 1–3. The Gaussian kernel is valid because $k_\sigma(\mathbf{x}, \mathbf{z}) = f(\mathbf{x}) \exp(\mathbf{x}^\mathsf{T}\mathbf{z}/\sigma^2) f(\mathbf{z})$ with $f(\mathbf{x}) = \exp(-\|\mathbf{x}\|^2 / 2\sigma^2)$: $\mathbf{x}^\mathsf{T}\mathbf{z}/\sigma^2$ is a kernel by rule 2, its exponential by rule 5, and the product with f by rule 4 — the d-dimensional version of Worked example 5.

6 Kernel ridge regression

Ridge regression (Note 8, Section 4) kernelises in a few lines and gives the simplest complete kernel method. Map the inputs, collect the features as the rows of $\Phi \in \mathbb{R}^{n\times D}$, and minimise

$$J(\mathbf{w}) = \|\Phi\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|^2 . \tag{7}$$

To keep the algebra short we drop the bias: centre the targets first (subtract their mean, add it back to predictions), or add a constant to the kernel. The primal solution is $\mathbf{w} = (\Phi^{\mathsf{T}}\Phi + \lambda I_D)^{-1}\Phi^{\mathsf{T}}\mathbf{y}$, which needs a $D \times D$ system — impossible for the Gaussian kernel, where $D = \infty$.

Derivation through the representer form. Set the gradient of (7) to zero:

$$2\Phi^{\mathsf{T}}(\Phi\mathbf{w} - \mathbf{y}) + 2\lambda\mathbf{w} = \mathbf{0} \quad\Longrightarrow\quad \lambda\mathbf{w} = \Phi^{\mathsf{T}}(\mathbf{y} - \Phi\mathbf{w}) \quad\Longrightarrow\quad \mathbf{w} = \Phi^{\mathsf{T}}\boldsymbol{\alpha}, \qquad \boldsymbol{\alpha} = \frac{1}{\lambda}(\mathbf{y} - \Phi\mathbf{w}).$$

This defines $\boldsymbol{\alpha}$ only implicitly, since $\mathbf{w}$ still appears on the right; the next step solves for it. Already, however, $\mathbf{w} = \sum_i \alpha_i \phi(\mathbf{x}_i)$, as the representer theorem promised, with α_i proportional to the residual of example i. (In regression there is no label factor: unlike the perceptron and the SVM, where $\mathbf{w} = \sum_i \alpha_i t_i \phi(\mathbf{x}_i)$, here $\mathbf{w} = \sum_i \alpha_i \phi(\mathbf{x}_i)$ and α_i carries its own sign.) Substitute $\mathbf{w} = \Phi^{\mathsf{T}}\boldsymbol{\alpha}$ into the definition of $\boldsymbol{\alpha}$ and use $\Phi\Phi^{\mathsf{T}} = K$ (the Gram matrix, $(\Phi\Phi^{\mathsf{T}})_{ij} = \phi(\mathbf{x}_i)^{\mathsf{T}}\phi(\mathbf{x}_j)$):

$$\lambda\boldsymbol{\alpha} = \mathbf{y} - \Phi\Phi^{\mathsf{T}}\boldsymbol{\alpha} = \mathbf{y} - K\boldsymbol{\alpha} \quad\Longrightarrow\quad (K + \lambda I_n)\,\boldsymbol{\alpha} = \mathbf{y} \quad\Longrightarrow\quad \boldsymbol{\alpha} = (K + \lambda I_n)^{-1}\mathbf{y}.$$

The prediction at a new point is

$$f(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}\phi(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i\, k(\mathbf{x}_i, \mathbf{x}) = \mathbf{k}(\mathbf{x})^{\mathsf{T}}\boldsymbol{\alpha}, \qquad \mathbf{k}(\mathbf{x}) = \big(k(\mathbf{x}_1, \mathbf{x}), \dots, k(\mathbf{x}_n, \mathbf{x})\big)^{\mathsf{T}}. \tag{8}$$

Only an $n \times n$ system is solved, whatever D.

The same result from a matrix identity. For any Φ and $\lambda > 0$, $\Phi^{\mathsf{T}}(\Phi\Phi^{\mathsf{T}} + \lambda I_n) = \Phi^{\mathsf{T}}\Phi\Phi^{\mathsf{T}} + \lambda\Phi^{\mathsf{T}} = (\Phi^{\mathsf{T}}\Phi + \lambda I_D)\Phi^{\mathsf{T}}$. Multiplying on the left by $(\Phi^{\mathsf{T}}\Phi + \lambda I_D)^{-1}$ and on the right by $(\Phi\Phi^{\mathsf{T}} + \lambda I_n)^{-1}$ gives the *push-through identity*

$$(\Phi^{\mathsf{T}}\Phi + \lambda I_D)^{-1}\Phi^{\mathsf{T}} = \Phi^{\mathsf{T}}(\Phi\Phi^{\mathsf{T}} + \lambda I_n)^{-1},$$

so the primal solution equals $\Phi^{\mathsf{T}}(K + \lambda I_n)^{-1}\mathbf{y} = \Phi^{\mathsf{T}}\boldsymbol{\alpha}$. A $D \times D$ inverse has been traded for an $n \times n$ one.

Worked example — 7. Kernel ridge regression on three points

Data: $x = 0, 1, 2$ with targets $\mathbf{y} = (1, 2, 0)$. Gaussian kernel with $\sigma = 1$, so $k(x, z) = e^{-(x-z)^2/2}$, and $\lambda = 0.1$. To show what happens far from the data we deliberately do *not* centre the targets at first; Algorithm 4 would, and the last paragraph shows the difference.
Gram matrix. $k(0,1) = k(1,2) = e^{-1/2} = 0.6065$ and $k(0,2) = e^{-2} = 0.1353$:

$$K = \begin{pmatrix} 1 & 0.6065 & 0.1353 \\ 0.6065 & 1 & 0.6065 \\ 0.1353 & 0.6065 & 1 \end{pmatrix}.$$

Coefficients and check. Adding 0.1 to the diagonal and solving $(K + 0.1\,I)\boldsymbol{\alpha} = \mathbf{y}$ (by elimination, or a computer) gives

$$\boldsymbol{\alpha} = (-0.546,\ 2.992,\ -1.582), \quad K\boldsymbol{\alpha} = (1.055,\ 1.701,\ 0.158), \quad \mathbf{y} - K\boldsymbol{\alpha} = (-0.055,\ 0.299,\ -0.158).$$

The middle point, whose target stands above its neighbours, receives a large positive weight; its neighbours receive negative weights that pull the curve back down. The residuals equal $\lambda\boldsymbol{\alpha}$, as the derivation requires: regularisation keeps the fit from passing through the points.
Prediction. At $x = 0.5$, $\mathbf{k} = (e^{-0.125}, e^{-0.125}, e^{-1.125}) = (0.8825, 0.8825, 0.3247)$, and

$$f(0.5) = 0.8825(-0.546) + 0.8825(2.992) + 0.3247(-1.582) = 1.645.$$

Similarly $f(1.5) = 1.066$, $f(3) = -0.561$ and $f(5) = -0.017$. Far from the data every $k(x_i, x) \to 0$, so the prediction returns to zero; the value at $x = 3$ undershoots below zero because of the negative coefficient of the last point.
With centring. Algorithm 4 first subtracts $\bar{y} = 1$, solves with $\mathbf{y} - \bar{y} = (0, 1, -1)$ and adds $\bar{y}$ back. This gives $\boldsymbol{\alpha} = (-1.338,\ 2.956,\ -2.374)$ and $f(0.5) = 1.657$, $f(3) = -0.055$, $f(5) = 0.975$: far from the data the prediction now returns to the mean of the targets, a more sensible default than zero.

What λ does. On the training points the fitted values are $\hat{\mathbf{y}} = K\boldsymbol{\alpha} = K(K + \lambda I)^{-1}\mathbf{y}$. With the eigendecomposition $K = \sum_j \mu_j \mathbf{u}_j \mathbf{u}_j^{\mathsf{T}}$ ($\mu_j \geq 0$ because K is PSD),

$$\hat{\mathbf{y}} = \sum_j \frac{\mu_j}{\mu_j + \lambda}\, \mathbf{u}_j\, \mathbf{u}_j^{\mathsf{T}}\mathbf{y},$$

the shrinkage formula of Note 8, Section 4, with μ_j in place of s_j^2 (for the linear kernel, $K = XX^{\mathsf{T}}$ and $\mu_j = s_j^2$ exactly). As $\lambda \to 0$, a model with an invertible K — the Gaussian kernel on distinct points — interpolates the training data. As λ grows, $\boldsymbol{\alpha} \approx \mathbf{y}/\lambda$ and $f(\mathbf{x}) \approx \frac{1}{\lambda}\sum_i y_i k(\mathbf{x}_i, \mathbf{x})$, a heavily shrunk, unnormalised version of the Parzen smoother of Note 5, Section 3.

Algorithm 4 Kernel ridge regression

Require: training inputs $\mathbf{x}_1, \dots, \mathbf{x}_n$, targets $\mathbf{y} \in \mathbb{R}^n$, kernel k, $\lambda > 0$

function KRRFit($X, \mathbf{y}; k, \lambda$)
  $\bar{y} \leftarrow \frac{1}{n}\sum_i y_i$; $\quad \mathbf{y} \leftarrow \mathbf{y} - \bar{y}\mathbf{1}$ ▷ centre the targets
  $K \leftarrow$ Gram($X; k$) ▷ $O(n^2 d)$
  $R \leftarrow$ Cholesky($K + \lambda I$) ▷ $K + \lambda I = RR^{\mathsf{T}}$, R lower triangular, $\approx n^3/3$ flops
  solve $R\mathbf{v} = \mathbf{y}$, then $R^{\mathsf{T}}\boldsymbol{\alpha} = \mathbf{v}$ ▷ two triangular solves, $O(n^2)$
  return $(\boldsymbol{\alpha}, \bar{y})$
end function
function KRRPredict($\mathbf{x}; \mathbf{x}_1, \dots, \mathbf{x}_n, \boldsymbol{\alpha}, \bar{y}, k$)
  $\mathbf{k} \leftarrow \big(k(\mathbf{x}_1, \mathbf{x}), \dots, k(\mathbf{x}_n, \mathbf{x})\big)$ ▷ $O(nd)$
  return $\bar{y} + \mathbf{k}^{\mathsf{T}}\boldsymbol{\alpha}$
end function

Running time — kernel ridge regression

Training: $O(n^2 d)$ for the Gram matrix plus $O(n^3)$ for the Cholesky factorisation, which dominates for large n: about 3×10^{11} floating-point operations for $n = 10^4$ (seconds on a laptop), 3×10^{14} for $n = 10^5$. *Memory*: $O(n^2)$. *Prediction*: $O(nd)$ per query; all α_i are non-zero in general, so every training point is used. Refitting for a new λ is cheap if one computes the eigendecomposition of K once ($O(n^3)$), after which each λ costs $O(n^2)$. *Compare* the primal ridge on explicit features: $O(nD^2 + D^3)$ training and $O(D)$ prediction. The kernel form wins when $D > n$, the primal form when $n > D$.

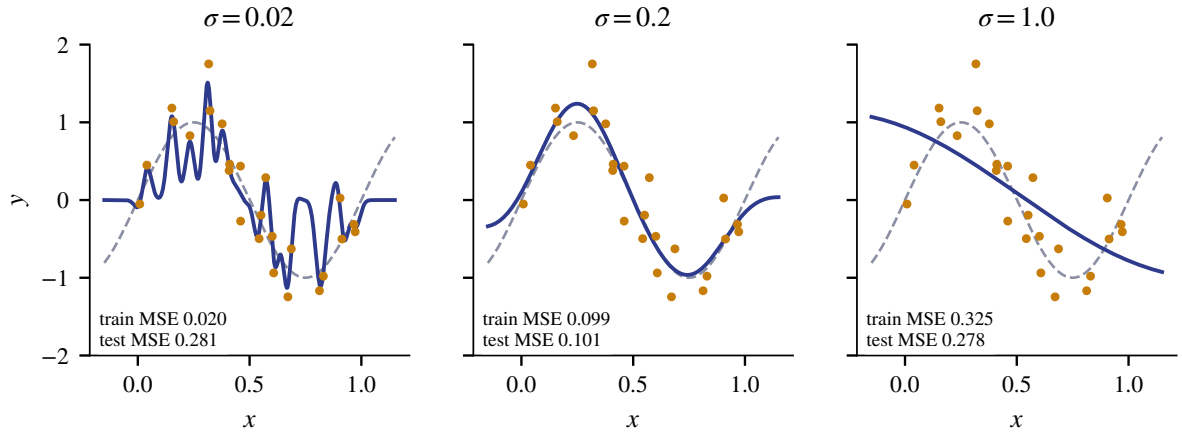


Figure 3. Kernel ridge regression with a Gaussian kernel and $\lambda = 0.1$ on 25 noisy samples (dots) of $\sin 2\pi x$ (dashed), for three widths, fitted without centring the targets. Errors are mean squared errors, the test error on 2000 new samples; the noise variance is 0.09, the smallest test error that any model can reach on average. A narrow kernel follows the noise and falls back to zero between the points; a wide kernel is too stiff to follow the sine.

Figure 3 shows the width at work in regression. With $\sigma = 0.02$ the training mean squared error is small (0.020) but the test error is 0.281; $\sigma = 0.2$ gives a test error of 0.101, close to the noise level 0.09; $\sigma = 1$ underfits (training 0.325, test 0.278). The width plays the role of the Parzen bandwidth (Note 5, Section 3) and λ that of the ridge penalty (Note 8, Section 4); together they set the capacity (Note 4, Section 5).

7 Kernel support vector machines

The SVM of Note 9, Section 5, chooses the maximum-margin hyperplane, and its dual problem (derived there from the Lagrangian, with the box $0 \le \alpha_i \le C$ of the soft margin) touches the data only through dot products $\mathbf{x}_i^{\mathsf{T}}\mathbf{x}_j$. In feature space each of them becomes $k(\mathbf{x}_i, \mathbf{x}_j)$, which gives the *kernel SVM* [6, 8]:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j t_i t_j \, k(\mathbf{x}_i, \mathbf{x}_j) \quad \text{subject to} \quad 0 \le \alpha_i \le C, \quad \sum_{i=1}^{n} \alpha_i t_i = 0, \tag{9}$$

with $C = \infty$ for the hard margin. Since K is PSD the objective is concave, so every local maximum is global (the maximiser is unique when K is positive definite). The decision function is

$$g(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i t_i \, k(\mathbf{x}_i, \mathbf{x}) + b \tag{10}$$

(unlike in (4), here g includes the bias).

Support vectors. At the optimum the Karush–Kuhn–Tucker conditions give three implications: $t_i g(\mathbf{x}_i) > 1$ (beyond the margin) $\Rightarrow \alpha_i = 0$; $0 < \alpha_i < C \Rightarrow t_i g(\mathbf{x}_i) = 1$ (exactly on the margin); $t_i g(\mathbf{x}_i) < 1$ (inside the margin or misclassified) $\Rightarrow \alpha_i = C$. Note 9 derived the hard-margin condition $\alpha_i[t_i g(\mathbf{x}_i) - 1] = 0$; for the soft margin, each slack $\xi_i \ge 0$ (at the optimum $\xi_i = \max(0, 1 - t_i g(\mathbf{x}_i))$) has its own multiplier $\mu_i \ge 0$. Setting the derivative of the Lagrangian with respect to ξ_i to zero gives $\alpha_i + \mu_i = C$, and complementary slackness gives $\alpha_i[t_i g(\mathbf{x}_i) - 1 + \xi_i] = 0$ and $\mu_i \xi_i = 0$. A point inside the margin has $\xi_i > 0$, so $\mu_i = 0$ and $\alpha_i = C$; a point strictly beyond it has $\xi_i = 0$ and $t_i g(\mathbf{x}_i) - 1 > 0$,

so $\alpha_i = 0$; and $0 < \alpha_i < C$ means $\mu_i > 0$, hence $\xi_i = 0$ and $t_i g(\mathbf{x}_i) = 1$. Points with $t_i g(\mathbf{x}_i) = 1$ may have any α_i in $[0, C]$. Only points with $\alpha_i > 0$, the *support vectors*, appear in (10); the others could be deleted without changing the classifier. The bias follows from any support vector s with $0 < \alpha_s < C$: $b = t_s - \sum_i \alpha_i t_i k(\mathbf{x}_i, \mathbf{x}_s)$ (in practice, averaged over all of them). In Figure 2 the good classifiers keep about 40 of the 200 points; the overfitting narrow kernel keeps 158 — many support vectors is a warning sign.

Worked example — 8. The kernel SVM on XOR

Return to the four XOR points and $k = (1 + \mathbf{x}^\mathsf{T}\mathbf{z})^2$, with $K = 8I + \mathbf{1}\mathbf{1}^\mathsf{T}$ and $\mathbf{t} = (1, 1, -1, -1)$ (hard margin, $C = \infty$). Because $K_{ij} = 8\delta_{ij} + 1$ and $t_i^2 = 1$, the double sum in (9) splits into two simple pieces:

$$\sum_{i,j} \alpha_i \alpha_j t_i t_j K_{ij} = 8 \sum_i \alpha_i^2 + \Big(\sum_i \alpha_i t_i \Big)^2,$$

and the constraint $\sum_i \alpha_i t_i = 0$ removes the second. On the feasible set the dual objective is therefore

$$\sum_i \alpha_i - 4 \sum_i \alpha_i^2 = \sum_i \big(\alpha_i - 4\alpha_i^2 \big),$$

a sum of separate parabolas. Each term is largest where its derivative $1 - 8\alpha_i$ vanishes, at $\alpha_i = 1/8$, and $\boldsymbol{\alpha} = \frac{1}{8}(1, 1, 1, 1)$ also satisfies $\sum_i \alpha_i t_i = \frac{1}{8}(1 + 1 - 1 - 1) = 0$. It maximises every term at once, so it is the optimum. The bias follows from point 1, a support vector with $0 < \alpha_1 < C$:

$$b = t_1 - \sum_i \alpha_i t_i K_{i1} = 1 - \tfrac{1}{8}(9 + 1 - 1 - 1) = 0,$$

and (10) gives

$$g(\mathbf{x}) = \tfrac{1}{8} \cdot 8\, x_1 x_2 = x_1 x_2,$$

using the sum computed in Worked example 4. Every point has $t_i g(\mathbf{x}_i) = 1$: all four are support vectors, on the margin. In feature space $\|\mathbf{w}\|^2 = \sum_{i,j} \alpha_i \alpha_j t_i t_j K_{ij} = 32/64 = \frac{1}{2}$, so the geometric margin is $1/\|\mathbf{w}\| = \sqrt{2}$ — the margin γ used in the mistake bound of Worked example 4. The perceptron's $8x_1x_2$ and the SVM's x_1x_2 have the same boundary here only because of the symmetry; in general the perceptron stops at any separator, the SVM at the one of largest margin.

Training by dual coordinate ascent. Absorb the bias into the kernel, $\tilde{k} = k + 1$ ("A bias for free"). The equality constraint $\sum_i \alpha_i t_i = 0$, which came from the derivative with respect to b, then disappears and each α_i can be improved on its own, subject only to $0 \le \alpha_i \le C$. The dual objective is now

$$Q(\boldsymbol{\alpha}) = \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j t_i t_j \tilde{K}_{ij}, \qquad \text{with scores } g_i = \sum_j \alpha_j t_j \tilde{K}_{ij}.$$

Change only α_i, to $\alpha_i + \delta$. The linear sum grows by δ; in the double sum only the terms with i as one (or both) of the indices change, the part linear in δ is $2\delta\, t_i \sum_j \alpha_j t_j \tilde{K}_{ij} = 2\delta\, t_i g_i$ (the factor 2 because i can be the first or the second index), and the part quadratic in δ, from the term (i, i), is $\delta^2 t_i^2 \tilde{K}_{ii} = \delta^2 \tilde{K}_{ii}$. Hence

$$Q(\boldsymbol{\alpha} + \delta\, \mathbf{e}_i) - Q(\boldsymbol{\alpha}) = \delta\,(1 - t_i g_i) - \tfrac{1}{2}\delta^2\, \tilde{K}_{ii},$$

a downward parabola in δ (its first and second derivatives at $\delta = 0$ are $\partial Q/\partial \alpha_i = 1 - t_i g_i$ and $-\tilde{K}_{ii}$) with its top at $\delta = (1 - t_i g_i)/\tilde{K}_{ii}$. Move there, clip to $[0, C]$, update all scores, and sweep repeatedly (Algorithm 5). Unlike for the perceptron, absorbing the bias slightly changes the problem: the bias is now the weight of a constant feature and is penalised like the other weights, so the SVM minimises

$\frac{1}{2}(\|\mathbf{w}\|^2 + b^2)$ instead of $\frac{1}{2}\|\mathbf{w}\|^2$. The two solutions usually differ only a little, and they coincide when the free-bias optimum has $b = 0$, as in Worked example 8. This scheme is a simplified relative of SMO [15], which keeps a separate bias and therefore changes two multipliers at a time.

Algorithm 5 Kernel SVM: training by dual coordinate ascent, and prediction

Require: training set $\{(\mathbf{x}_i, t_i)\}_{i=1}^n$, kernel k, $C > 0$, number of sweeps E

function KernelSVMFit($\{(\mathbf{x}_i, t_i)\}$; k, C, E)
  $\tilde{K} \leftarrow \textsc{Gram}(X; k) + \mathbf{1}\mathbf{1}^\mathsf{T}$ ▷ bias absorbed, $O(n^2 d)$
  $\boldsymbol{\alpha} \leftarrow \mathbf{0}$; $\mathbf{g} \leftarrow \mathbf{0}$ ▷ $g_i = \sum_j \alpha_j t_j \tilde{K}_{ij}$
  for sweep $= 1, \dots, E$ **do**
    for $i = 1, \dots, n$ in random order **do**
      $a \leftarrow \min\big(C, \max(0,\ \alpha_i + (1 - t_i g_i)/\tilde{K}_{ii})\big)$ ▷ $O(1)$
      $\mathbf{g} \leftarrow \mathbf{g} + (a - \alpha_i)\, t_i\, \tilde{K}_{:,i}$; $\alpha_i \leftarrow a$ ▷ $O(n)$
    end for
  end for
  return $\boldsymbol{\alpha}$ ▷ support vectors: $\alpha_i > 0$
end function
function KernelSVMPredict($\mathbf{x}$; $\{(\mathbf{x}_s, t_s, \alpha_s)\}_{\alpha_s > 0}$, k)
  return $\operatorname{sign}\big(\sum_s \alpha_s t_s\,[k(\mathbf{x}_s, \mathbf{x}) + 1]\big)$ ▷ $O(n_{\mathrm{SV}}\, d)$
end function

Running time — kernel SVM

Training: $O(n^2 d)$ for the Gram matrix, then $O(n)$ per coordinate update and $O(En^2)$ for E sweeps. Generic quadratic-programming solvers cost $O(n^3)$; SMO, used in the LIBSVM library, needs between $O(n^2)$ and $O(n^3)$ kernel evaluations ($O(n^2 d)$ to $O(n^3 d)$ time), depending on C and on the data. *Memory*: $O(n^2)$, or a cache of the most used rows of K. *Prediction*: $O(n_{\mathrm{SV}}\, d)$ per query.

Worked example — 9. Dual coordinate ascent on XOR, by hand

Run Algorithm 5 on the XOR data of Worked example 8, with C so large that it never clips and the fixed order $1, 2, 3, 4$. With the bias absorbed, $\tilde{K} = K + \mathbf{1}\mathbf{1}^\mathsf{T}$ has 10 on the diagonal and 2 elsewhere, so each update is $\alpha_i \leftarrow \alpha_i + (1 - t_i g_i)/10$, after which the score vector changes by (change of α_i) t_i times column i of $\tilde{K}$. The first sweep, starting from $\boldsymbol{\alpha} = \mathbf{0}$, $\mathbf{g} = \mathbf{0}$:

i	$t_i g_i$ before	new α_i	$(t_1 g_1, t_2 g_2, t_3 g_3, t_4 g_4)$ after
1	0	0.1	$(1,\ 0.2,\ -0.2,\ -0.2)$
2	0.2	0.08	$(1.16,\ 1,\ -0.36,\ -0.36)$
3	−0.36	0.136	$(0.888,\ 0.728,\ 1,\ -0.088)$
4	−0.088	0.1088	$(0.670,\ 0.510,\ 1.218,\ 1)$

Each update puts its own point exactly on the margin ($t_i g_i = 1$), but moves the others: after sweep 1, $\boldsymbol{\alpha} = (0.1, 0.08, 0.136, 0.1088)$. After sweep 2, $\boldsymbol{\alpha} = (0.133, 0.122, 0.129, 0.125)$ and the margins are $(1.065, 0.981, 1.033, 1)$; after sweep 5 every α_i agrees with $1/8$ to four decimals and every $t_i g_i$ lies within 0.0003 of 1. The iteration has found the analytic optimum of Worked example 8. Absorbing the bias did not change it, because there $\sum_i \alpha_i t_i = 0$: the absorbed bias $b = \sum_i \alpha_i t_i$ is zero at the optimum, as was the free one.

8 The kernel family

Kernelisable algorithms. Many linear algorithms can be written with dot products only, and each has a kernel version. They differ in what the coefficients mean, whether most of them vanish, and what training costs (all start with the $O(n^2 d)$ Gram matrix; T is a number of passes or iterations, E a number of sweeps):

algorithm	coefficient of example i	sparse?	training beyond K
perceptron (Section 3)	number of mistakes on i	yes	$O(Tn^2)$
SVM (Section 7)	Lagrange multiplier of the margin constraint	yes	$O(En^2)$ to $O(n^3)$
ridge regression (Section 6)	residual of i, divided by λ	no	$O(n^3)$
logistic regression (Algorithm 6)	weight β_i in $g = \sum_i \beta_i k(\mathbf{x}_i, \cdot)$	no	$O(Tn^2)$ by gradient descent
support vector regression	multiplier of the ε-tube constraint	yes	as the SVM
Gaussian process	as ridge, λ = noise variance	no	$O(n^3)$; error bar $O(n^2)$ per query
kernel PCA	component of an eigenvector of centred K	no	$O(n^3)$ eigendecomposition

Plain (unregularised) least squares is also in the family: it is the limit $\lambda \to 0$ of kernel ridge regression, $K\boldsymbol{\alpha} = \mathbf{y}$, which for an invertible K — the Gaussian kernel on distinct points — interpolates the training data (Section 6, "What λ does"); that is why $\lambda > 0$ is used in practice.

Kernel logistic regression uses $0/1$ labels as in Note 9, Section 4, written here $y_i = (1 + t_i)/2 \in \{0, 1\}$, the score $g(\mathbf{x}) = \sum_i \beta_i k(\mathbf{x}_i, \mathbf{x})$ (a bias absorbed as in "A bias for free") and the probability $\sigma(g)$, with $\sigma(a) = 1/(1 + e^{-a})$. On the training points the scores are $\mathbf{g} = K\boldsymbol{\beta}$, and it minimises the cross-entropy plus the penalty $\lambda \boldsymbol{\beta}^{\mathsf{T}} K \boldsymbol{\beta}$, which equals $\lambda \|\mathbf{w}\|^2$ for $\mathbf{w} = \sum_i \beta_i \phi(\mathbf{x}_i)$:

$$J(\boldsymbol{\beta}) = -\sum_i \big[y_i \log \sigma(g_i) + (1 - y_i) \log(1 - \sigma(g_i)) \big] + \lambda \boldsymbol{\beta}^{\mathsf{T}} K \boldsymbol{\beta}, \qquad \nabla J = K\big(\sigma(K\boldsymbol{\beta}) - \mathbf{y}\big) + 2\lambda K \boldsymbol{\beta}.$$

The gradient follows from the chain rule: the cross-entropy has derivative $\sigma(g_i) - y_i$ with respect to g_i (Note 9, Section 4), and $\partial g_i / \partial \beta_j = K_{ij}$. There is no closed form, so the minimiser is found by gradient descent (Note 7), Algorithm 6. It gives probabilities but, unlike the SVM, keeps every training point.

Algorithm 6 Kernel logistic regression by gradient descent

Require: training set $\{(\mathbf{x}_i, t_i)\}_{i=1}^n$, kernel k, $\lambda > 0$, step size η, number of iterations T

function KLRFit($\{(\mathbf{x}_i, t_i)\}$; k, λ, η, T)
  $K \leftarrow \textsc{Gram}(X; k) + \mathbf{1}\mathbf{1}^{\mathsf{T}}$; $y_i \leftarrow (1 + t_i)/2$ ▷ bias absorbed, $O(n^2 d)$
  $\boldsymbol{\beta} \leftarrow \mathbf{0}$
  for iteration $= 1, \dots, T$ **do**
    $\mathbf{r} \leftarrow \sigma(K\boldsymbol{\beta}) - \mathbf{y}$ ▷ residuals of the probabilities, $O(n^2)$
    $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} - \eta\, K(\mathbf{r} + 2\lambda\boldsymbol{\beta})$ ▷ gradient step, $O(n^2)$
  end for
  return $\boldsymbol{\beta}$
end function
function KLRPredict($\mathbf{x}$; $\{(\mathbf{x}_i, \beta_i)\}$, k)
  return $P(t = +1 \mid \mathbf{x}) = \sigma\big(\sum_i \beta_i \,[k(\mathbf{x}_i, \mathbf{x}) + 1]\big)$ ▷ $O(nd)$
end function

Running time — kernel logistic regression

Training: $O(n^2 d)$ for the Gram matrix, then two matrix–vector products with K, $O(n^2)$, per iteration: $O(n^2 d + Tn^2)$ in total (Newton's method would converge in fewer iterations but costs $O(n^3)$ per step). *Memory*: $O(n^2)$ for K. *Prediction*: $O(nd)$ per query, since all β_i are non-zero in general.

The remaining rows of the table are mentioned for orientation and are not developed here. Support vector regression ignores errors smaller than ε, which is what makes it sparse. A *Gaussian process* [17] is the Bayesian version of ridge regression (Note 8, Section 6): its predictive mean is exactly (8) with λ equal to the noise variance, and it adds an error bar; in geostatistics the same method is called kriging. Kernel PCA diagonalises the centred Gram matrix and finds non-linear directions of variation [19]; projecting a new point on q components costs $O(nd + nq)$. Algorithms that look at single coordinates rather than dot products, such as decision trees (Note 3, Section 4), do not kernelise.

Two meanings of "kernel". Note 5, Section 3, used a kernel as a *smoothing window* — written $K(\mathbf{x}_i, \mathbf{x})$ there, $w(\mathbf{x}_i, \mathbf{x})$ here to avoid a clash with the Gram matrix: a non-negative weight, decreasing with distance, used in a weighted average $\sum_i w\, y_i / \sum_i w$ with no training at all. This note's kernels are *Mercer kernels*: dot products in a feature space, with PSD Gram matrices, whose coefficients α_i are learned. The requirements differ:

	smoothing kernel (Note 5)	Mercer kernel (this note)
requirement	non-negative, peaked at $\mathbf{x} = \mathbf{z}$, decreasing with distance; integrates to 1 for density estimation	symmetric, every Gram matrix PSD
prediction	$\sum_i w(\mathbf{x}_i, \mathbf{x})\, y_i \big/ \sum_i w(\mathbf{x}_i, \mathbf{x})$	$\sum_i \alpha_i\, k(\mathbf{x}_i, \mathbf{x})$, $\boldsymbol{\alpha}$ fitted
training	none	solve for $\boldsymbol{\alpha}$
examples	Gaussian, box window, Epanechnikov	Gaussian, polynomial, linear, Laplacian

The Gaussian is both, which is why the word is shared. The other entries are not interchangeable. The polynomial kernel $(1 + \mathbf{x}^\mathsf{T}\mathbf{z})^3$ is a valid Mercer kernel but a useless smoothing window: it grows with distance and can be negative. The box window $\mathbf{1}\{|x - z| < 1\}$ is a fine smoothing window but not a Mercer kernel: on the points $0, 0.6, 1.2$ its Gram matrix is

$$\begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix}, \qquad \mathbf{c} = (1, -1, 1) : \quad \mathbf{c}^\mathsf{T} K \mathbf{c} = 3 - 2 - 2 + 0 = -1 < 0,$$

with eigenvalues $1 - \sqrt{2}$, 1 and $1 + \sqrt{2}$. Note also that Section 6 links the two worlds: for large λ, kernel ridge regression becomes a Parzen-like weighted sum of the targets.

9 Running time and scaling to many examples

Table 2 collects the costs, with n examples in d dimensions, D explicit features, T passes or E sweeps, and n_{SV} support vectors.

Which is cheaper. The explicit route pays for the dimension D of the feature space; the kernel route pays for the number of examples, through the $n \times n$ Gram matrix. Kernels win when D is huge or infinite and n is moderate — up to some 10^4 to 10^5 examples, a regime common in physics, where each example may be an expensive simulation or measurement. Explicit or learned features win when n is in the millions, where an $n \times n$ matrix can neither be stored nor factorised.

Table 2. Costs of explicit and kernel methods. Kernel evaluations are assumed to cost $O(d)$; D is the number of features of whichever explicit map is used (polynomial or random Fourier).

method	training time	memory	prediction / query
explicit map + perceptron or GD	$O(nD + TnD)$	$O(nD)$	$O(D)$
explicit map + ridge (primal)	$O(nD^2 + D^3)$	$O(nD + D^2)$	$O(D)$
Gram matrix	$O(n^2 d)$	$O(n^2)$	—
kernel perceptron	$O(n^2 d + Tn^2)$	$O(n^2)$	$O(Md)$, $M \le n$
kernel ridge regression	$O(n^2 d + n^3)$	$O(n^2)$	$O(nd)$
kernel SVM	$O(n^2 d + En^2)$; SMO $O(n^2 d)$ to $O(n^3 d)$	$O(n^2)$ or cache	$O(n_{\mathrm{SV}}\, d)$
kernel logistic regression	$O(n^2 d + Tn^2)$	$O(n^2)$	$O(nd)$
random Fourier features $\hat{\phi}$ + ridge	$O(nDd + nD^2 + D^3)$	$O(nD)$	$O(Dd)$

Random Fourier features. For large n one can return to an explicit map that *approximates* a given kernel with a modest number D of features [16] (D now counts random features). For the Gaussian kernel the construction takes a few lines. Let $\boldsymbol{\omega} \sim \mathcal{N}(\mathbf{0}, \sigma^{-2} I)$ and $\tau \sim \mathrm{Uniform}[0, 2\pi]$ (a random phase). For a fixed vector $\mathbf{u}$, $Y = \boldsymbol{\omega}^{\mathsf{T}}\mathbf{u}$ is a one-dimensional centred Gaussian with variance $\rho^2 = \|\mathbf{u}\|^2/\sigma^2$, and a centred Gaussian has $\mathbb{E}[\cos Y] = \exp(-\rho^2/2)$. To see this, write $\cos Y$ as the real part of e^{iY} and complete the square in the exponent, $iy - y^2/2\rho^2 = -(y - i\rho^2)^2/2\rho^2 - \rho^2/2$:

$$\mathbb{E}[e^{iY}] = \frac{1}{\sqrt{2\pi\rho^2}} \int_{-\infty}^{\infty} e^{iy - y^2/2\rho^2}\, dy = e^{-\rho^2/2}\, \frac{1}{\sqrt{2\pi\rho^2}} \int_{-\infty}^{\infty} e^{-(y - i\rho^2)^2/2\rho^2}\, dy = e^{-\rho^2/2},$$

because the last integral, a Gaussian shifted parallel to the real axis, still equals $\sqrt{2\pi\rho^2}$. (Without complex numbers: expand $\cos Y = \sum_m (-1)^m Y^{2m}/(2m)!$ and use $\mathbb{E}[Y^{2m}] = (2m-1)(2m-3)\cdots 1 \cdot \rho^{2m}$, which gives the series of $e^{-\rho^2/2}$ term by term.) So $\mathbb{E}[\cos(\boldsymbol{\omega}^{\mathsf{T}}\mathbf{u})] = \exp(-\|\mathbf{u}\|^2/2\sigma^2)$. The identity $2\cos A\cos B = \cos(A-B) + \cos(A+B)$ with $A = \boldsymbol{\omega}^{\mathsf{T}}\mathbf{x} + \tau$, $B = \boldsymbol{\omega}^{\mathsf{T}}\mathbf{z} + \tau$ then gives

$$\mathbb{E}\big[2\cos(\boldsymbol{\omega}^{\mathsf{T}}\mathbf{x} + \tau)\cos(\boldsymbol{\omega}^{\mathsf{T}}\mathbf{z} + \tau)\big] = \mathbb{E}\big[\cos(\boldsymbol{\omega}^{\mathsf{T}}(\mathbf{x} - \mathbf{z}))\big] + \underbrace{\mathbb{E}\big[\cos(\boldsymbol{\omega}^{\mathsf{T}}(\mathbf{x} + \mathbf{z}) + 2\tau)\big]}_{0} = k_\sigma(\mathbf{x}, \mathbf{z}),$$

where the second term averages to zero because 2τ runs uniformly over two whole periods of the cosine. Averaging over D independent draws $(\boldsymbol{\omega}_j, \tau_j)$, the random-feature map $\hat{\phi}(\mathbf{x})$ with components $\hat{\phi}_j(\mathbf{x}) = \sqrt{2/D}\,\cos(\boldsymbol{\omega}_j^{\mathsf{T}}\mathbf{x} + \tau_j)$ satisfies $\hat{\phi}(\mathbf{x})^{\mathsf{T}}\hat{\phi}(\mathbf{z}) \approx k_\sigma(\mathbf{x}, \mathbf{z})$. Its root-mean-square error is $\sqrt{(1 - k_\sigma^2 + k_\sigma^4/2)/D}$, writing k_σ for $k_\sigma(\mathbf{x}, \mathbf{z})$: one term $2\cos A\cos B$ has mean k_σ and mean square $\mathbb{E}[\cos^2(\boldsymbol{\omega}^{\mathsf{T}}(\mathbf{x} - \mathbf{z}))] + \mathbb{E}[\cos^2(\boldsymbol{\omega}^{\mathsf{T}}(\mathbf{x} + \mathbf{z}) + 2\tau)] = \frac{1}{2}(1 + k_\sigma^4) + \frac{1}{2}$ (the cross term averages to zero over τ), hence variance $1 - k_\sigma^2 + k_\sigma^4/2$, and averaging D independent terms divides the variance by D. For $\mathbf{x} = (1, 0)$, $\mathbf{z} = (0, 1)$ and $\sigma = 1$, where $k_\sigma = e^{-1} = 0.368$, this is $0.93/\sqrt{D}$: 0.30, 0.093, 0.030 and 0.009 for $D = 10$, 100, 1000 and 10 000 (a simulation with 1000 draws agrees to within 3%). A linear method on $\hat{\phi}(\mathbf{x})$ then costs time linear in n. The random draw is made once and reused: training and test points must be mapped with the same Ω and $\boldsymbol{\tau}$ (Algorithm 7). A second remedy, the Nyström method, approximates K from m randomly chosen columns at cost $O(nmd + nm^2)$ [22]; like the other methods mentioned only for orientation, it is not developed here.

Running time — random Fourier features

Features: $O(nDd)$ time and $O(nD)$ memory. *Ridge on top*: $O(nD^2 + D^3)$, linear in n; with $D = 10^3$ this handles millions of examples. *Prediction*: $O(Dd)$ per query, independent of n. The price is an approximation error of order $1/\sqrt{D}$ in each kernel value.

Algorithm 7 Random Fourier features for the Gaussian kernel

Require: input dimension d, width σ, number of features D

function RFFDraw(d; σ, D) ▷ called once, before training
  draw $\Omega \in \mathbb{R}^{D\times d}$ with entries $\mathcal{N}(0, \sigma^{-2})$ and $\boldsymbol{\tau} \in [0, 2\pi]^D$ uniformly
  return $(\Omega, \boldsymbol{\tau})$
end function
function RFFMap(X; $\Omega, \boldsymbol{\tau}$) ▷ same $(\Omega, \boldsymbol{\tau})$ for training and test data
  $\hat{\Phi} \leftarrow \sqrt{2/D}\,\cos\left(X\Omega^{\mathsf{T}} + \mathbf{1}\boldsymbol{\tau}^{\mathsf{T}}\right)$ ▷ $n \times D$, rows $\hat{\phi}(\mathbf{x}_i)^{\mathsf{T}}$, $O(nDd)$
  return $\hat{\Phi}$ ▷ then fit ridge or a linear classifier on $\hat{\Phi}$
end function

Modern connection — wide neural networks are kernel machines

Random features are a neural network whose first layer is frozen at random values and whose last layer is trained. Note 4, Section 8, showed double descent for exactly such a model. At $D \approx n$, the interpolation threshold, the minimum-norm solution sits at the double-descent peak; only as D grows far beyond n ($D \to \infty$) does it approach the minimum-norm interpolant (kernel regression with $\lambda \to 0$) of the kernel that the features approximate [4] — for random ReLU features, the arc-cosine kernel [7]. Jacot, Gabriel and Hongler showed that something similar holds for the *whole* network: with the standard (NTK) scaling of the weights, as every layer becomes infinitely wide, the *neural tangent kernel* $\Theta(\mathbf{x}, \mathbf{z}) = \nabla_{\boldsymbol{\theta}} f(\mathbf{x})^{\mathsf{T}} \nabla_{\boldsymbol{\theta}} f(\mathbf{z})$ becomes deterministic and stays constant during training, so gradient descent on the network is kernel gradient descent with a fixed kernel [11]. In this regime wide networks are kernel methods; conversely, kernel machines that interpolate their training data show the same surprisingly good generalisation as over-parametrised networks [3]. Practical networks are not in this limit: their features move during training, the third route of Section 2, and this feature learning is a leading explanation of why they beat fixed kernels on images and text. Week 5 starts there.

In physics — kernel regression of potential-energy surfaces

Quantum-mechanical calculations of the energy and forces of a configuration of atoms are accurate but slow; molecular dynamics needs millions of them. Bartók, Payne, Kondor and Csányi introduced *Gaussian approximation potentials* (GAP) [2]: the total energy is written as a sum of local atomic energies, each a Gaussian-process (that is, kernel) regression on a descriptor of the atom's neighbourhood that is invariant under rotations, translations and permutations of identical atoms. Such potentials approach quantum-mechanical accuracy at a small fraction of the cost. Rupp, Tkatchenko, Müller and von Lilienfeld similarly used kernel ridge regression on a matrix of Coulomb repulsions between nuclei to predict the atomisation energies of several thousand small organic molecules [18]. In both cases the physics enters through the kernel: what does it mean for two atomic environments to be similar?

Summary A fixed non-linear feature map ϕ turns any linear method into a non-linear one; explicit polynomial maps have $\binom{d+p}{p}$ features and quickly become too large. Algorithms that use the data only through dot products — the perceptron, ridge regression, the SVM, logistic regression, PCA — can instead be written with coefficients α_i, one per training example, and a kernel $k(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^{\mathsf{T}}\phi(\mathbf{z})$ that is cheap to evaluate even when ϕ is infinite-dimensional, as for the Gaussian kernel. A function is a valid kernel exactly when its Gram matrices are symmetric positive semi-definite; sums, positive multiples and products of kernels are kernels. The kernel

perceptron counts mistakes, kernel ridge regression solves $(K + \lambda I)\boldsymbol{\alpha} = \mathbf{y}$, and the kernel SVM keeps only its support vectors. The kernel width and the regularisation constant set the capacity and are chosen by validation. The price is an $n \times n$ Gram matrix: $O(n^2)$ memory and up to $O(n^3)$ time, which random features or learned features avoid when n is large.

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. Explain how a linear classifier trained on $\phi(\mathbf{x})$ can have a curved decision boundary in the input space. For the quadratic map $\phi(x_1, x_2) = (x_1, x_2, x_1^2, x_1 x_2, x_2^2)$, which shapes of boundary are possible?

A2. In the centred coordinates $s_j = 2x_j - 1$ of Section 1, the class of each XOR point is the sign of $s_1 s_2$. Explain why adding this single feature makes the points linearly separable, and describe the resulting boundary in the original (x_1, x_2) plane.

A3. Name the three routes to a non-linear feature map discussed in Section 2. Give one advantage and one disadvantage of each.

A4. True or false, with a justification: "The kernel trick is a fast method for computing $\phi(\mathbf{x})$."

A5. Explain in words why adding to $\mathbf{w}$ a component orthogonal to all the $\phi(\mathbf{x}_i)$ can only increase a ridge-regularised objective. What does this imply about how many numbers are needed to describe the solution when the feature space is infinite-dimensional?

A6. In the kernel perceptron, what does α_i count? What does $\alpha_i = 0$ say about example i, and why do such examples cost nothing at prediction time?

A7. A student trains a linear perceptron on the raw inputs, obtains $\mathbf{w} = \sum_i \alpha_i t_i \mathbf{x}_i$, and then uses $\phi(\mathbf{w})$ as the weight vector in feature space. Explain why this is not the kernel perceptron.

A8. Describe what happens to the Gram matrix and to the decision boundary of a Gaussian-kernel classifier as $\sigma \to 0$ and as $\sigma \to \infty$. Which regime overfits? Relate your answer to Figure 2.

A9. Why must every Gram matrix of a valid kernel be positive semi-definite? Explain the argument in terms of the length of a vector in feature space.

A10. Without expanding any feature map, decide which of these are valid kernels on $\mathbb{R}^d$ and justify each answer with a rule of Section 5 or a counter-example: (a) $3\,\mathbf{x}^\mathsf{T}\mathbf{z}$; (b) $-\mathbf{x}^\mathsf{T}\mathbf{z}$; (c) $(\mathbf{x}^\mathsf{T}\mathbf{z})^2 + 1$; (d) $\|\mathbf{x} - \mathbf{z}\|^2$; (e) $k_\sigma(\mathbf{x}, \mathbf{z})\,(1 + \mathbf{x}^\mathsf{T}\mathbf{z})^2$.

A11. In kernel ridge regression, what happens to the fit as $\lambda \to 0$ and as $\lambda \to \infty$? Why does the prediction of a Gaussian-kernel model return to zero far from the data, and why does this make centring the targets important?

A12. Which training points are support vectors of a kernel SVM? Why does the prediction cost depend on their number, and what might a very large number of support vectors indicate?

A13. Compare the kernel perceptron, kernel ridge regression and the kernel SVM with respect to how many coefficients α_i are non-zero after training.

A14. The Gaussian function appears in both Note 5 (Parzen windows) and this note. Explain the different roles it plays. Give one function that is a good smoothing window but not a Mercer kernel, and one Mercer kernel that is not a useful smoothing window.

A15. You have $n = 10^6$ examples with $d = 20$ features. Explain why exact kernel ridge regression is impractical, and describe two alternatives.

A16. For ridge regression with a polynomial kernel, when is the explicit (primal) form cheaper to train than kernel ridge regression? Answer in terms of n and D.

A17. Why should the input features be standardised before a Gaussian kernel is used? Give an example of what goes wrong otherwise.

A18. True or false, with a justification: "A kernel method has no hyperparameters, because the kernel is chosen in advance."

A19. Random Fourier features are a neural network whose first layer is frozen at random values. What is trained in such a model, and what does a practical neural network learn that a fixed-kernel method cannot?

B. Short calculations

B1. How many monomials of degree at most 2 are there in $d = 3$ variables? List them. How many of degree at most 4 in $d = 20$ variables?

B2. For $\mathbf{x} = (2, -1)$ and $\mathbf{z} = (1, 3)$ compute $(\mathbf{x}^\mathsf{T}\mathbf{z})^2$ directly, and again as $\phi(\mathbf{x})^\mathsf{T}\phi(\mathbf{z})$ with the map $\phi(\mathbf{x}) = (x_1^2, \sqrt{2}\,x_1x_2, x_2^2)$. Then do the same for $(1 + \mathbf{x}^\mathsf{T}\mathbf{z})^2$ with the six-dimensional map of Worked example 3.

B3. Run the kernel perceptron (Algorithm 2) on the one-dimensional data $x = -2, -1, 1, 2$ with labels $t = +1, -1, -1, +1$ and the kernel $k(x, z) = (1+xz)^2$, visiting the points in this order. First compute the 4×4 Gram matrix. Then carry out the first two passes and report $\boldsymbol{\alpha}$ after each. (Continued, the algorithm makes no mistake in its fourth pass and stops with $\boldsymbol{\alpha} = (1, 3, 3, 1)$.) Using this final $\boldsymbol{\alpha}$, write the decision function $g(x)$ as a polynomial in x. Where is the decision boundary? Compare with the explicit route of Worked example 2.

B4. Show that $\begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$ cannot be the Gram matrix of any kernel by exhibiting a vector $\mathbf{c}$ with $\mathbf{c}^\mathsf{T}K\mathbf{c} < 0$. Then compute the Gram matrix of the Gaussian kernel with $\sigma = 1$ on the points $x = 0, 1, 3$ (four decimals) and check that its leading principal minors (the determinants of its top-left 1×1, 2×2 and 3×3 sub-matrices) are positive.

B5. Kernel ridge regression with two points: $x_1 = 0$, $x_2 = 1$, targets $\mathbf{y} = (1, -1)$, a Gaussian kernel with $k(0, 1) = 0.5$, and $\lambda = 0.5$. Compute $\boldsymbol{\alpha} = (K + \lambda I)^{-1}\mathbf{y}$, the fitted values at the two training points, and the predictions at $x = 0.5$ and $x = -0.5$, using $k(0, \pm 0.5) = k(1, 0.5) = 0.841$ and $k(1, -0.5) = 0.210$.

B6. A kernel SVM with $k(x, z) = (1 + xz)^2$ trained on $x = -1, 0, 1$ with labels $t = +1, -1, +1$ returned $\boldsymbol{\alpha} = (1, 2, 1)$ and $b = -1$. Check that $\sum_i \alpha_i t_i = 0$, write $g(x)$ as a polynomial, evaluate

it at $x = 0.5$ and $x = 2$, verify that all three points lie on the margin, and find the decision boundary.

B7. For the one-dimensional Gaussian kernel $k(x, z) = \exp(-(x - z)^2/2\sigma^2)$, compute $\partial k/\partial x$. At which distances $|x - z|$ does the kernel decrease fastest? Interpret σ as a length scale.

B8. You have $n = 5\times10^4$ examples in $d = 100$ dimensions and compare kernel ridge regression with the kernel $k(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}^\mathsf{T}\mathbf{z})^2$ against primal ridge regression on its $D = \binom{102}{2}$ explicit features (constant included). Compute D and estimate (a) the memory of the Gram matrix in 64-bit floats, (b) the number of operations to compute it ($\approx n^2 d$), (c) the number of operations for the Cholesky factorisation in kernel ridge regression ($\approx n^3/3$), (d) the memory of the explicit features in 64-bit floats, and (e) the number of operations of the primal solution, $\approx nD^2 + D^3$. Which approach would you choose?

C. Derivations

C1. Show that $(1 + \mathbf{x}^\mathsf{T}\mathbf{z})^2 = \phi(\mathbf{x})^\mathsf{T}\phi(\mathbf{z})$ for $\mathbf{x}, \mathbf{z} \in \mathbb{R}^d$ with an explicit ϕ, and check that its dimension is $\binom{d+2}{2}$. Then prove (2) by the dummy-variable argument of Section 2.

C2. Prove the push-through identity and use it to derive $\boldsymbol{\alpha} = (K+\lambda I)^{-1}\mathbf{y}$ for kernel ridge regression. Show that the fitted values are $K(K + \lambda I)^{-1}\mathbf{y}$, and that the fit interpolates the data as $\lambda \to 0$ when K is invertible.

C3. Prove that the sum, the positive multiple and the product of two kernels are kernels, by constructing their feature maps. Deduce that the Gaussian kernel is valid in any dimension.

C4. Starting from the linear SVM dual of Note 9, Section 5, explain which quantities change when the inputs are replaced by $\phi(\mathbf{x}_i)$, and obtain (9). Show that keeping a bias in the kernel perceptron is equivalent to replacing k by $k + 1$. Then, for the SVM with $\tilde{k} = k + 1$ and no separate bias, derive the coordinate update of Algorithm 5 from the first and second derivatives of the dual objective with respect to α_i.

D. Programming

D1. Implement Algorithms 3 and 2 in numpy and run the kernel perceptron on noisy two-moons data with a cubic polynomial kernel and Gaussian kernels of several widths. Plot the decision regions, and report the number of mistakes and the number of non-zero α_i. What happens when the data are not separable in feature space?

D2. Implement kernel ridge regression (Algorithm 4) and reproduce Figure 3 (the figure was fitted without centring the targets, so with centring your curves will differ slightly, mostly near the ends). Choose σ and λ by 5-fold cross-validation and compare the result with polynomial ridge regression (Note 8) of the best degree.

D3. Implement random Fourier features (Algorithm 7). (a) Plot the error of $\hat{\phi}(\mathbf{x})^\mathsf{T}\hat{\phi}(\mathbf{z})$ as an approximation of $k_\sigma(\mathbf{x}, \mathbf{z})$ against D on log–log axes and confirm the $1/\sqrt{D}$ slope. (b) For n from 10^3 to 2×10^4, time exact kernel ridge regression and ridge regression on $D = 500$ random features, and compare their test errors.

Further reading. Bishop [5], chapters 6 and 7; Hastie, Tibshirani and Friedman [10], sections 5.8 and 12.3; Murphy [14], chapter 17; Schölkopf and Smola [21] for the theory of kernels; Rasmussen

and Williams [17] for Gaussian processes, the probabilistic reading of kernel ridge regression.

References

[1] M. A. Aizerman, E. M. Braverman and L. I. Rozonoer, Theoretical foundations of the potential function method in pattern recognition learning, *Autom. Remote Control* 25, 821–837 (1964).

[2] A. P. Bartók, M. C. Payne, R. Kondor and G. Csányi, Gaussian approximation potentials: the accuracy of quantum mechanics, without the electrons, *Phys. Rev. Lett.* 104, 136403 (2010).

[3] M. Belkin, S. Ma and S. Mandal, To understand deep learning we need to understand kernel learning, *ICML* (2018).

[4] M. Belkin, D. Hsu, S. Ma and S. Mandal, Reconciling modern machine-learning practice and the classical bias–variance trade-off, *Proc. Natl. Acad. Sci. USA* 116, 15849–15854 (2019).

[5] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[6] B. E. Boser, I. M. Guyon and V. N. Vapnik, A training algorithm for optimal margin classifiers, *Proc. 5th Annual Workshop on Computational Learning Theory (COLT)*, 144–152 (1992).

[7] Y. Cho and L. K. Saul, Kernel methods for deep learning, *NeurIPS* (2009).

[8] C. Cortes and V. Vapnik, Support-vector networks, *Mach. Learn.* 20, 273–297 (1995).

[9] Y. Freund and R. E. Schapire, Large margin classification using the perceptron algorithm, *Mach. Learn.* 37, 277–296 (1999).

[10] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer (2009).

[11] A. Jacot, F. Gabriel and C. Hongler, Neural tangent kernel: convergence and generalization in neural networks, *NeurIPS* (2018).

[12] G. Kimeldorf and G. Wahba, Some results on Tchebycheffian spline functions, *J. Math. Anal. Appl.* 33, 82–95 (1971).

[13] J. Mercer, Functions of positive and negative type, and their connection with the theory of integral equations, *Phil. Trans. R. Soc. Lond. A* 209, 415–446 (1909).

[14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press (2022).

[15] J. C. Platt, Sequential minimal optimization: a fast algorithm for training support vector machines, Microsoft Research Technical Report MSR-TR-98-14 (1998).

[16] A. Rahimi and B. Recht, Random features for large-scale kernel machines, *NeurIPS* (2007).

[17] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press (2006).

[18] M. Rupp, A. Tkatchenko, K.-R. Müller and O. A. von Lilienfeld, Fast and accurate modeling of molecular atomization energies with machine learning, *Phys. Rev. Lett.* 108, 058301 (2012).

[19] B. Schölkopf, A. Smola and K.-R. Müller, Nonlinear component analysis as a kernel eigenvalue problem, *Neural Comput.* 10, 1299–1319 (1998).

[20] B. Schölkopf, R. Herbrich and A. J. Smola, A generalized representer theorem, *Proc. COLT*, 416–426 (2001).

[21] B. Schölkopf and A. J. Smola, *Learning with Kernels*, MIT Press (2002).

[22] C. K. I. Williams and M. Seeger, Using the Nyström method to speed up kernel machines, *NeurIPS* (2001).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.